

A
Design and Analysis of Algorithms Course PBL Report on

LONGEST PATH FINDER
submitted in partial fulfillment of the requirements for the award of the Degree of
BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING - DATA SCIENCE

by

24R11A6717 - J.V.Saaketh
24R11A6722 - K.Rajkumar
24R11A6724 - M.Sundar

Under the esteemed guidance of
Mr.Pandu Ranga Thota

Professor



Department of
Computer Science & Engineering –Data Science

GEETHANJALI COLLEGE OF ENGINEERING & TECHNOLOGY
(GCET)

Approved by AICTE, New Delhi, Affiliated to JNTUH University, Hyderabad and Accredited by NAAC & NBA.
Cheeryal(V), Keesara(M), Medchal(Dist.), Telangana – 501 301.
(An Autonomous Institution)

April,2026

**GEETHANJALI COLLEGE OF ENGINEERING & TECHNOLOGY
(GCET)**

Approved by AICTE, New Delhi, Affiliated to JNTUH University, Hyderabad and Accredited by NAAC & NBA.
Cheeryal(V), Keesara(M), Medchal(Dist.), Telangana – 501 301.

(An Autonomous Institution)

DEPARTMENT OF CSE - DATA SCIENCE



CERTIFICATE

This is to certify that the B.Tech PBL Project report entitled “**Longest Path Finder:Using Dijkstra’s Algorithm**” is a Bonafide work done by **J.V.Saaketh** bearing **24R11A6717** , **K.RajKumar** bearing **24R11A6722** and **M.Sundar** bearing **24R11A6724** in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering – Data Science** from Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

Internal Guide
Mr.Pandu Ranga Thota
Professor

HOD, CSE-DS
Dr. D. Sujatha
Professor

External Examiner

ABSTRACT

This project implements a Longest Path Finder to determine the maximum distance between two nodes in a graph. Unlike shortest path problems, finding the longest path is computationally complex and is generally NP-Hard in graphs containing cycles.

To overcome this, the system uses Directed Acyclic Graph (DAG)-based approach and modified Dijkstra techniques (using maximum priority). The application provides a graphical interface to select start and end nodes and visualizes the longest path along with cost and explored nodes.

The project highlights the difference between shortest and longest path problems and demonstrates practical strategies to solve them efficiently under constraints.

TABLE OF CONTENTS

	Page No
1. Methodology	1
2. Software and Hardware Requirements	2
3. Algorithm	3
4. Flowchart	4
5. Key Differences.	5
6. Code	6-23
7. Output Screens	24-25
8. Conclusion	26
9. Reference	27

METHODOLOGY

Graph Representation

- Nodes represent locations
- Edges represent paths with weight

Can be:

- Directed graph (preferred)
- DAG for guaranteed correctness

Longest Path Algorithm (DAG Approach)

- Perform Topological Sorting
- Initialize distances to $-\infty$
- Set source distance = 0
- Relax edges in topological order:
$$\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + \text{weight})$$

Modified Dijkstra Approach (Alternative)

- Use Max Priority Queue
- Instead of minimizing distance $\rightarrow$ maximize it

Works only if:

- No positive cycles
- Or constraints applied

Path Construction

- Parent array used
- Backtrack from destination to source

Visualization

- Graph drawn using Java Swing
- Longest path highlighted
- Nodes explored shown dynamically

SOFTWARE AND HARDWARE REQUIREMENTS

Software

- Java (JDK 8+)
- Swing (GUI)
- IDE: IntelliJ / Eclipse / VS Code

Hardware

- Processor: Intel Pentium or above
- RAM: Minimum 2 GB
- Storage: 100 MB free
- System Type: 32-bit or 64-bit

ALGORITHM

Longest Path in DAG

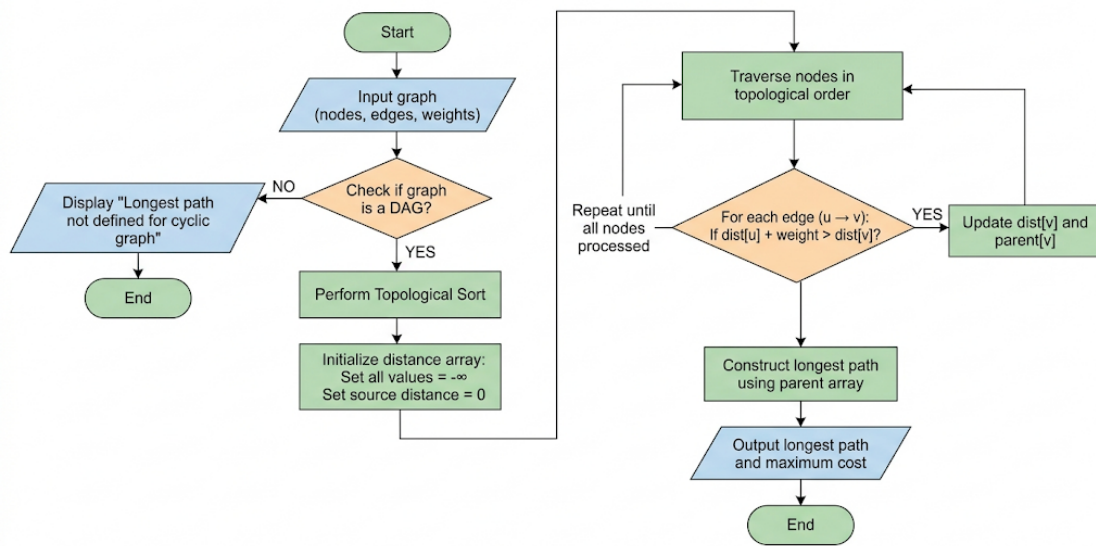
1. Perform Topological Sort of graph
2. Initialize $\text{dist}[] = -\infty$
3. $\text{dist}[\text{source}] = 0$
4. For each node u in topological order:
 For each neighbor v :
 if $\text{dist}[u] + \text{weight} > \text{dist}[v]$:
 $\text{dist}[v] = \text{dist}[u] + \text{weight}$
 $\text{parent}[v] = u$
5. Construct path using $\text{parent}[]$

Time Complexity

- Topological Sort $\rightarrow O(V + E)$
- Relaxation $\rightarrow O(V + E)$
- Total $\rightarrow O(V + E)$

FLOWCHART

Longest Path Finder using Directed Acyclic Graph (DAG)



KEY DIFFERENCES

Feature	Shortest Path	Longest Path
Objective	Minimize cost	Maximize Cost
Complexity	Polynomial	NP-Hard
Algorithm	Dijkstra	DAG
Cycles	Allowed	Not Allowed

CODE

LongestPathAlgorithm

```
package com.example.shortestpathfinder.algorithm;

import com.example.shortestpathfinder.model.Graph;

import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * Finds the longest simple path between two cities using DFS/backtracking.
 *
 * <p>In an undirected positive-weight graph, a longest path is only finite when
 * cities are not revisited. This class therefore searches for the longest simple
 * path from source to destination.</p>
 */
public class LongestPathAlgorithm {

    public PathResult findLongestPath(Graph graph, String source, String destination) {
        if (graph == null) {
            throw new IllegalArgumentException("Graph cannot be null.");
        }
        if (source == null || source.trim().isEmpty() || destination == null ||
destination.trim().isEmpty()) {
            return PathResult.failure("Please select both source and destination cities.");
        }

        String normalizedSource = source.trim();
        String normalizedDestination = destination.trim();

        if (!graph.containsCity(normalizedSource) || !
graph.containsCity(normalizedDestination)) {
            return PathResult.failure("Selected city does not exist in the graph.");
        }
        if (normalizedSource.equalsIgnoreCase(normalizedDestination)) {
            return PathResult.success(List.of(normalizedSource), 0, "Longest simple path
found.");
        }

        BestPath bestPath = new BestPath();
        Set<String> visited = new HashSet<>();
        List<String> currentPath = new ArrayList<>();

        visited.add(normalizedSource);
        currentPath.add(normalizedSource);
```

```

        search(graph, normalizedSource, normalizedDestination, visited, currentPath, 0,
bestPath);

        if (bestPath.path.isEmpty()) {
            return PathResult.failure("No path exists between " + normalizedSource + " and
" + normalizedDestination + ".");
        }

        return PathResult.success(bestPath.path, bestPath.totalDistance, "Longest simple
path found.");
    }

    private void search(Graph graph, String currentCity, String destination, Set<String>
visited,
                        List<String> currentPath, long distance, BestPath bestPath) {
        if (currentCity.equals(destination)) {
            if (distance > bestPath.totalDistance) {
                bestPath.path = List.copyOf(currentPath);
                bestPath.totalDistance = distance;
            }
            return;
        }

        for (Graph.Edge edge : graph.getNeighbors(currentCity)) {
            if (visited.contains(edge.destination())) {
                continue;
            }

            visited.add(edge.destination());
            currentPath.add(edge.destination());

            search(graph, edge.destination(), destination, visited, currentPath,
distance + edge.distance(), bestPath);

            currentPath.remove(currentPath.size() - 1);
            visited.remove(edge.destination());
        }
    }

    private static class BestPath {
        private List<String> path = List.of();
        private long totalDistance = Long.MIN_VALUE;
    }
}

```

PathResult

```
package com.example.shortestpathfinder.algorithm;

import java.util.Collections;
import java.util.List;

/**
 * Holds the outcome of a path search.
 */
public class PathResult {
    private final boolean pathExists;
    private final List<String> path;
    private final long totalDistance;
    private final String message;

    private PathResult(boolean pathExists, List<String> path, long totalDistance, String
message) {
        this.pathExists = pathExists;
        this.path = List.copyOf(path);
        this.totalDistance = totalDistance;
        this.message = message;
    }

    public static PathResult success(List<String> path, long totalDistance) {
        return new PathResult(true, path, totalDistance, "Path found.");
    }

    public static PathResult success(List<String> path, long totalDistance, String
message) {
        return new PathResult(true, path, totalDistance, message);
    }

    public static PathResult failure(String message) {
        return new PathResult(false, Collections.emptyList(), -1, message);
    }

    public boolean pathExists() {
        return pathExists;
    }

    public List<String> getPath() {
        return path;
    }

    public long getTotalDistance() {
        return totalDistance;
    }

    public String getMessage() {
        return message;
    }
}
```

Graph

```
package com.example.shortestpathfinder.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;

/**
 * Represents an undirected weighted graph using an adjacency list.
 */
public class Graph {
    private final Map<String, List<Edge>> adjacencyList = new LinkedHashMap<>();

    /**
     * Creates a deep copy of the graph so algorithms can read a stable snapshot.
     *
     * @return copied graph
     */
    public Graph copy() {
        Graph copy = new Graph();
        for (Map.Entry<String, List<Edge>> entry : adjacencyList.entrySet()) {
            List<Edge> edges = new ArrayList<>();
            for (Edge edge : entry.getValue()) {
                edges.add(new Edge(edge.destination(), edge.distance()));
            }
            copy.adjacencyList.put(entry.getKey(), edges);
        }
        return copy;
    }

    /**
     * Adds a city if it does not already exist.
     *
     * @param city city name
     */
    public void addCity(String city) {
        validateCityName(city);
        adjacencyList.putIfAbsent(city.trim(), new ArrayList<>());
    }

    /**
     * Adds an undirected edge between two cities.
     *
     * @param source source city
```

```

* @param target destination city
* @param distance positive distance between cities
*/
public void addBidirectionalEdge(String source, String target, int distance) {
    validateCityName(source);
    validateCityName(target);

    if (source.trim().equalsIgnoreCase(target.trim())) {
        throw new IllegalArgumentException("Source and destination cities must be
different.");
    }
    if (distance <= 0) {
        throw new IllegalArgumentException("Distance must be greater than zero.");
    }

    String normalizedSource = source.trim();
    String normalizedTarget = target.trim();

    addCity(normalizedSource);
    addCity(normalizedTarget);

    addOrReplaceDirectedEdge(normalizedSource, normalizedTarget, distance);
    addOrReplaceDirectedEdge(normalizedTarget, normalizedSource, distance);
}

public boolean containsCity(String city) {
    return city != null && adjacencyList.containsKey(city.trim());
}

public List<String> getCities() {
    List<String> cities = new ArrayList<>(adjacencyList.keySet());
    cities.sort(Comparator.naturalOrder());
    return Collections.unmodifiableList(cities);
}

public List<Edge> getNeighbors(String city) {
    if (!containsCity(city)) {
        return List.of();
    }
    return Collections.unmodifiableList(adjacencyList.get(city.trim()));
}

private void addOrReplaceDirectedEdge(String source, String target, int distance) {
    List<Edge> neighbors = adjacencyList.get(source);
    neighbors.removeIf(edge -> edge.destination().equalsIgnoreCase(target));
    neighbors.add(new Edge(target, distance));
}

```

```

private void validateCityName(String city) {
    if (city == null || city.trim().isEmpty()) {
        throw new IllegalArgumentException("City name cannot be empty.");
    }
}

/**
 * Immutable edge record for an adjacency list entry.
 *
 * @param destination neighboring city
 * @param distance    edge weight
 */
public record Edge(String destination, int distance) {
    public Edge {
        Objects.requireNonNull(destination, "Destination city cannot be null.");
        if (destination.trim().isEmpty()) {
            throw new IllegalArgumentException("Destination city cannot be empty.");
        }
        if (distance <= 0) {
            throw new IllegalArgumentException("Distance must be greater than zero.");
        }
    }
}
}

```

PathFinderFrame

```
package com.example.shortestpathfinder.ui;

import com.example.shortestpathfinder.algorithm.LongestPathAlgorithm;
import com.example.shortestpathfinder.algorithm.PathResult;
import com.example.shortestpathfinder.model.Graph;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingWorker;
import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.Point;
import java.awt.RenderingHints;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.stream.Collectors;

/**
 * Swing UI for finding and visualizing the longest simple path in a city graph.
 */
public class PathFinderFrame extends JFrame {
    private static final int MAP_WIDTH = 760;
    private static final int MAP_HEIGHT = 460;

    private final Graph graph;
    private final LongestPathAlgorithm longestPathAlgorithm;
```

```

private final Object graphLock = new Object();
private final JComboBox<String> sourceComboBox = new JComboBox<>();
private final JComboBox<String> destinationComboBox = new JComboBox<>();
private final JComboBox<String> connectToComboBox = new JComboBox<>();
private final JButton findButton = new JButton("Find Longest Path");
private final JButton addCityButton = new JButton("Add City");
private final JLabel pathLabel = new JLabel("Longest Path: -");
private final JLabel distanceLabel = new JLabel("Total Distance: -");
private final JLabel statusLabel = new JLabel("Choose two cities, then click Find
Longest Path.");
private final JTextArea routesArea = new JTextArea(9, 30);
private final JTextField newCityField = new JTextField();
private final JTextField distanceField = new JTextField();
private final GraphPanel graphPanel = new GraphPanel();

public PathfinderFrame(Graph graph, LongestPathAlgorithm longestPathAlgorithm) {
    this.graph = Objects.requireNonNull(graph, "Graph cannot be null.");
    this.longestPathAlgorithm = Objects.requireNonNull(longestPathAlgorithm,
"LongestPathAlgorithm cannot be null.");

    setTitle("Longest Path Finder using Swing");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setMinimumSize(new Dimension(1120, 760));

    initializeView();
    refreshCitySelectors();
    updateRoutesSummary();
    graphPanel.setGraph(graph, List.of());
    pack();
    setLocationRelativeTo(null);
}

private void initializeView() {
    JPanel rootPanel = new JPanel(new BorderLayout(18, 18));
    rootPanel.setBorder(BorderFactory.createEmptyBorder(18, 18, 18, 18));
    rootPanel.setBackground(new Color(238, 243, 247));

    rootPanel.add(buildHeaderPanel(), BorderLayout.NORTH);
    rootPanel.add(buildContentPanel(), BorderLayout.CENTER);
    setContentPane(rootPanel);
}

private JPanel buildHeaderPanel() {
    JPanel headerPanel = new JPanel(new BorderLayout());
    headerPanel.setOpaque(false);

    JLabel titleLabel = new JLabel("Longest Path Finder");
    titleLabel.setFont(new Font("Verdana", Font.BOLD, 26));

```

```

        JLabel subtitleLabel = new JLabel("DFS/backtracking longest simple path with
Swing components");
        subtitleLabel.setForeground(new Color(72, 97, 116));

        JPanel titlePanel = new JPanel(new BorderLayout(0, 4));
        titlePanel.setOpaque(false);
        titlePanel.add(titleLabel, BorderLayout.NORTH);
        titlePanel.add(subtitleLabel, BorderLayout.SOUTH);

        headerPanel.add(titlePanel, BorderLayout.WEST);
        return headerPanel;
    }

    private JPanel buildContentPanel() {
        JPanel contentPanel = new JPanel(new BorderLayout(18, 0));
        contentPanel.setOpaque(false);

        contentPanel.add(buildControlsPanel(), BorderLayout.WEST);
        contentPanel.add(buildRightPanel(), BorderLayout.CENTER);

        return contentPanel;
    }

    private JPanel buildControlsPanel() {
        JPanel controlsPanel = createCardPanel(new GridBagLayout());
        controlsPanel.setPreferredSize(new Dimension(340, 620));

        GridBagConstraints constraints = baseConstraints();
        constraints.gridx = 0;
        constraints.gridy = 0;
        controlsPanel.add(createSectionTitle("Route Search"), constraints);

        addLabeledControl(controlsPanel, "Source City", sourceComboBox, 1);
        addLabeledControl(controlsPanel, "Destination City", destinationComboBox, 3);

        constraints.gridy = 5;
        findButton.addActionListener(event -> handleFindLongestPath());
        controlsPanel.add(findButton, constraints);

        constraints.gridy = 6;
        JPanel resultPanel = createCardPanel(new GridBagLayout());
        resultPanel.setBackground(new Color(255, 250, 241));
        addResultLabels(resultPanel);
        controlsPanel.add(resultPanel, constraints);

        constraints.gridy = 7;
        controlsPanel.add(createSectionTitle("Add New City"), constraints);
    }

```

```

addLabeledControl(controlsPanel, "City Name", newCityField, 8);
addLabeledControl(controlsPanel, "Connect To", connectToComboBox, 10);
addLabeledControl(controlsPanel, "Distance", distanceField, 12);

constraints.gridy = 14;
addCityButton.addActionListener(event -> handleAddCity());
controlsPanel.add(addCityButton, constraints);

return controlsPanel;
}

private JPanel buildRightPanel() {
    JPanel rightPanel = new JPanel(new BorderLayout(0, 16));
    rightPanel.setOpaque(false);

    JPanel mapCard = createCardPanel(new BorderLayout(0, 12));
    mapCard.add(createSectionTitle("City Map"), BorderLayout.NORTH);
    mapCard.add(graphPanel, BorderLayout.CENTER);

    routesArea.setEditable(false);
    routesArea.setLineWrap(true);
    routesArea.setWrapStyleWord(true);
    routesArea.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 13));

    JPanel routesCard = createCardPanel(new BorderLayout(0, 12));
    routesCard.add(createSectionTitle("Available Routes"), BorderLayout.NORTH);
    routesCard.add(new JScrollPane(routesArea), BorderLayout.CENTER);

    rightPanel.add(mapCard, BorderLayout.CENTER);
    rightPanel.add(routesCard, BorderLayout.SOUTH);
    return rightPanel;
}

private void addResultLabels(JPanel resultPanel) {
    GridBagConstraints constraints = baseConstraints();
    constraints.insets = new Insets(4, 4, 4, 4);

    pathLabel.setFont(pathLabel.getFont().deriveFont(Font.BOLD));
    distanceLabel.setFont(distanceLabel.getFont().deriveFont(Font.BOLD));
    statusLabel.setForeground(new Color(72, 97, 116));

    constraints.gridy = 0;
    resultPanel.add(pathLabel, constraints);
    constraints.gridy = 1;
    resultPanel.add(distanceLabel, constraints);
    constraints.gridy = 2;
    resultPanel.add(statusLabel, constraints);
}

```

```

private void addLabeledControl(JPanel panel, String labelText, java.awt.Component
component, int startRow) {
    GridBagConstraints constraints = baseConstraints();
    constraints.gridy = startRow;
    panel.add(new JLabel(labelText), constraints);

    constraints.gridy = startRow + 1;
    panel.add(component, constraints);
}

private JLabel createSectionTitle(String text) {
    JLabel label = new JLabel(text);
    label.setFont(new Font("Verdana", Font.BOLD, 16));
    return label;
}

private JPanel createCardPanel(java.awt.LayoutManager layoutManager) {
    JPanel panel = new JPanel(layoutManager);
    panel.setBackground(Color.WHITE);
    panel.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(new Color(215, 228, 237)),
        BorderFactory.createEmptyBorder(14, 14, 14, 14)));
    return panel;
}

private GridBagConstraints baseConstraints() {
    GridBagConstraints constraints = new GridBagConstraints();
    constraints.gridx = 0;
    constraints.fill = GridBagConstraints.HORIZONTAL;
    constraints.weightx = 1;
    constraints.insets = new Insets(6, 0, 6, 0);
    return constraints;
}

private void handleFindLongestPath() {
    String source = (String) sourceComboBox.getSelectedItemAt();
    String destination = (String) destinationComboBox.getSelectedItemAt();
    setSearchInProgress(true);
    SwingWorker<PathResult, Void> worker = new SwingWorker<>() {
        @Override
        protected PathResult doInBackground() {
            Graph graphSnapshot;
            synchronized (graphLock) {
                graphSnapshot = graph.copy();
            }
            return longestPathAlgorithm.findLongestPath(graphSnapshot, source,
destination);
        }
    }
}

```

```

@Override
protected void done() {
    try {
        applyPathResult(get());
    } catch (Exception exception) {
        showError("Unable to compute the longest path.",
exception.getMessage());
    } finally {
        setSearchInProgress(false);
    }
}
};

worker.execute();
}

private void handleAddCity() {
    try {
        String newCity = newCityField.getText() == null ? "" :
newCityField.getText().trim();
        String connectTo = (String) connectToComboBox.getSelectedItem();
        String distanceText = distanceField.getText() == null ? "" :
distanceField.getText().trim();
        if (newCity.isEmpty() || connectTo == null || connectTo.isBlank() ||
distanceText.isEmpty()) {
            throw new IllegalArgumentException("Please enter a city name, a city to
connect to, and a valid distance.");
        }
        if (graph.containsCity(newCity)) {
            throw new IllegalArgumentException("That city already exists.");
        }
        int distance = Integer.parseInt(distanceText);
        synchronized (graphLock) {
            graph.addBidirectionalEdge(newCity, connectTo, distance);
        }
        newCityField.setText("");
        distanceField.setText("");
        refreshCitySelectors();
        updateRoutesSummary();
        graphPanel.setGraph(graph, List.of());
        statusLabel.setForeground(new Color(22, 101, 52));
        statusLabel.setText("Added city " + newCity + " and connected it to " +
connectTo + ".");
    } catch (NumberFormatException exception) {
        showError("Invalid Distance", "Distance must be a whole number.");
    } catch (IllegalArgumentException exception) {

```

```

showError("Cannot Add City", exception.getMessage());

```

```

    }
}

```

```

private void refreshCitySelectors() {
    String currentSource = (String) sourceComboBox.getSelectedItem();
    String currentDestination = (String) destinationComboBox.getSelectedItem();
    String currentConnectTo = (String) connectToComboBox.getSelectedItem();
    List<String> cities = graph.getCities();

```

```

        resetComboBox(sourceComboBox, cities, currentSource != null ? currentSource :
firstCity(cities));
        resetComboBox(destinationComboBox, cities,
            currentDestination != null ? currentDestination : (cities.size() > 1 ?
cities.get(1) : firstCity(cities)));
        resetComboBox(connectToComboBox, cities, currentConnectTo != null ?
currentConnectTo : firstCity(cities));
    }

```

```

private void resetComboBox(JComboBox<String> comboBox, List<String> cities,
String preferredValue) {
    comboBox.removeAllItems();
    for (String city : cities) {
        comboBox.addItem(city);
    }
    if (preferredValue != null && cities.contains(preferredValue)) {
        comboBox.setSelectedItem(preferredValue);
    } else if (!cities.isEmpty()) {
        comboBox.setSelectedIndex(0);
    }
}

```

```

private String firstCity(List<String> cities) {
    return cities.isEmpty() ? null : cities.get(0);
}

```

```

private void updateRoutesSummary() {
    StringBuilder summary = new StringBuilder();
    for (String city : graph.getCities()) {
        String routes = graph.getNeighbors(city).stream()
            .map(edge -> edge.destination() + " (" + edge.distance() + " km)")
            .collect(Collectors.joining(", "));
        summary.append(city).append(" ->
").append(routes).append(System.lineSeparator());
    }
    routesArea.setText(summary.toString());
}

```

18

```

private void applyPathResult(PathResult result) {
    if (result.pathExists()) {

```

```

        pathLabel.setText("Longest Path: " + String.join(" -> ", result.getPath()));
        distanceLabel.setText("Total Distance: " + result.getTotalDistance() + " km");
        statusLabel.setForeground(new Color(22, 101, 52));
        statusLabel.setText(result.getMessage());
        graphPanel.setGraph(graph, result.getPath());
        return;
    }

    pathLabel.setText("Longest Path: -");
    distanceLabel.setText("Total Distance: -");
    statusLabel.setForeground(new Color(185, 28, 28));
    statusLabel.setText(result.getMessage());
    graphPanel.setGraph(graph, List.of());
}

private void setSearchInProgress(boolean inProgress) {
    findButton.setEnabled(!inProgress);
    addCityButton.setEnabled(!inProgress);
    sourceComboBox.setEnabled(!inProgress);
    destinationComboBox.setEnabled(!inProgress);
    connectToComboBox.setEnabled(!inProgress);
    newCityField.setEnabled(!inProgress);
    distanceField.setEnabled(!inProgress);

    if (inProgress) {
        statusLabel.setForeground(new Color(15, 23, 42));
        statusLabel.setText("Searching for the longest simple path...");
    }
}

private void showError(String title, String message) {
    statusLabel.setForeground(new Color(185, 28, 28));
    statusLabel.setText(message);
    JOptionPane.showMessageDialog(this, message, title,
JOptionPane.ERROR_MESSAGE);
}

private class GraphPanel extends JPanel {
    private final Map<String, Point> cityPositions = new HashMap<>();
    private Graph graph;
    private List<String> highlightedPath = List.of();
    private GraphPanel() {
        setPreferredSize(new Dimension(MAP_WIDTH, MAP_HEIGHT));
        setBackground(new Color(251, 253, 255));
        setBorder(BorderFactory.createLineBorder(new Color(215, 228, 237)));
    }

    private void setGraph(Graph graph, List<String> highlightedPath) {
        this.graph = graph;

```

```

        this.highlightedPath = List.copyOf(highlightedPath);
        assignCityPositions();
        repaint();
    }

    @Override
    protected void paintComponent(Graphics graphics) {
        super.paintComponent(graphics);
        if (graph == null) {
            return;
        }

        Graphics2D graphics2D = (Graphics2D) graphics.create();
        graphics2D.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
        drawEdges(graphics2D);
        drawCities(graphics2D);
        graphics2D.dispose();
    }

    private void assignCityPositions() {
        if (graph == null) {
            return;
        }

        List<String> cities = graph.getCities();
        cityPositions.keySet().retainAll(cities);

        for (int index = 0; index < cities.size(); index++) {
            String city = cities.get(index);
            double angle = (2 * Math.PI * index) / Math.max(cities.size(), 1) - Math.PI / 2;
            int centerX = MAP_WIDTH / 2;
            int centerY = MAP_HEIGHT / 2;
            int radiusX = 275;
            int radiusY = 160;
            int x = (int) Math.round(centerX + radiusX * Math.cos(angle));
            int y = (int) Math.round(centerY + radiusY * Math.sin(angle));
            cityPositions.put(city, new Point(x, y));
        }
    }

    private void drawEdges(Graphics2D graphics2D) {
        List<String> drawnEdges = new ArrayList<>();
        for (String source : graph.getCities()) {
            for (Graph.Edge edge : graph.getNeighbors(source)) {
                String key = createEdgeKey(source, edge.destination());

```

```

        if (drawnEdges.contains(key)) {
            continue;
        }

        Point sourcePoint = cityPositions.get(source);
        Point destinationPoint = cityPositions.get(edge.destination());
        boolean highlighted = isEdgeInPath(source, edge.destination());

        graphics2D.setColor(highlighted ? new Color(239, 68, 68) : new Color(148,
163, 184));
        graphics2D.setStroke(new BasicStroke(highlighted ? 4f : 2f));
        graphics2D.drawLine(sourcePoint.x, sourcePoint.y, destinationPoint.x,
destinationPoint.y);

        graphics2D.setColor(new Color(72, 97, 116));
        graphics2D.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        int labelX = (sourcePoint.x + destinationPoint.x) / 2;
        int labelY = (sourcePoint.y + destinationPoint.y) / 2;
        graphics2D.drawString(edge.distance() + " km", labelX, labelY);

        drawnEdges.add(key);
    }
}
}
private void drawCities(Graphics2D graphics2D) {
    for (String city : graph.getCities()) {
        Point point = cityPositions.get(city);
        boolean highlighted = highlightedPath.contains(city);

        graphics2D.setColor(highlighted ? new Color(249, 115, 22) : new Color(14,
165, 233));
        graphics2D.fillOval(point.x - 24, point.y - 24, 48, 48);
        graphics2D.setColor(Color.WHITE);
        graphics2D.setStroke(new BasicStroke(3f));
        graphics2D.drawOval(point.x - 24, point.y - 24, 48, 48);

        graphics2D.setFont(new Font("Verdana", Font.BOLD, 12));
        FontMetrics metrics = graphics2D.getFontMetrics();
        int textX = point.x - metrics.stringWidth(city) / 2;
        int textY = point.y + metrics.getAscent() / 2 - 2;
        graphics2D.drawString(city, textX, textY);
    }
}
private boolean isEdgeInPath(String source, String destination) {
    for (int index = 0; index < highlightedPath.size() - 1; index++) {

```

```
String current = highlightedPath.get(index);
```

```

        String next = highlightedPath.get(index + 1);
        if ((current.equals(source) && next.equals(destination))
            || (current.equals(destination) && next.equals(source))) {
            return true;
        }
    }
    return false;
}

private String createEdgeKey(String cityOne, String cityTwo) {
    return cityOne.compareTo(cityTwo) <= 0 ? cityOne + "-" + cityTwo : cityTwo + "-"
+ cityOne;
}
}
}

```

LongestPathFinderApp

```
package com.example.shortestpathfinder;

import com.example.shortestpathfinder.algorithm.LongestPathAlgorithm;
import com.example.shortestpathfinder.model.Graph;
import com.example.shortestpathfinder.ui.PathFinderFrame;

import javax.swing.SwingUtilities;

/**
 * Entry point for the Swing Longest Path Finder application.
 */
public class LongestPathFinderApp {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            Graph graph = createSampleGraph();
            PathFinderFrame frame = new PathFinderFrame(graph, new
LongestPathAlgorithm());
            frame.setVisible(true);
        });
    }

    private static Graph createSampleGraph() {
        Graph graph = new Graph();

        // Sample dataset with weighted roads.
        graph.addBidirectionalEdge("Delhi", "Mumbai", 1400);
        graph.addBidirectionalEdge("Delhi", "Jaipur", 280);
        graph.addBidirectionalEdge("Jaipur", "Ahmedabad", 660);
        graph.addBidirectionalEdge("Ahmedabad", "Mumbai", 530);
        graph.addBidirectionalEdge("Delhi", "Lucknow", 555);
        graph.addBidirectionalEdge("Lucknow", "Varanasi", 320);
        graph.addBidirectionalEdge("Varanasi", "Kolkata", 680);
        graph.addBidirectionalEdge("Mumbai", "Bengaluru", 980);
        graph.addBidirectionalEdge("Ahmedabad", "Bengaluru", 1490);
        graph.addBidirectionalEdge("Kolkata", "Bengaluru", 1870);

        return graph;
    }
}
```

OUTPUT SCREENS

Longest Path Finder

DFS/backtracking longest simple path with Swing components

Route Search

Source City
Ahmedabad

Destination City
Bengaluru

Find Longest Path

Longest Path: -
Total Distance: -
Added city Hyderabad and connected it t...

Add New City

City Name

Connect To
Delhi

Distance

Add City

City Map

Available Routes

Bengaluru -> Mumbai (980 km), Ahmedabad (1490 km), Kolkata (1870 km)
Delhi -> Mumbai (1400 km), Jaipur (280 km), Lucknow (555 km), Hyderabad (1700 km)
Hyderabad -> Delhi (1700 km)
Jaipur -> Delhi (280 km), Ahmedabad (660 km)
Kolkata -> Varanasi (680 km), Bengaluru (1870 km)
Lucknow -> Delhi (555 km), Varanasi (320 km)
Mumbai -> Delhi (1400 km), Ahmedabad (530 km), Bengaluru (980 km)
Varanasi -> Lucknow (320 km), Kolkata (680 km)

Longest Path Finder

DFS/backtracking longest simple path with Swing components

Route Search

Source City
Jaipur

Destination City
Hyderabad

Find Longest Path

Longest Path: Jaipur -> Ahmedabad -> ...
Total Distance: 7295 km
Longest simple path found.

Add New City

City Name
Jaipur

Connect To
Hyderabad

Distance
2000

Add City

City Map

Available Routes

Bengaluru -> Mumbai (980 km), Ahmedabad (1490 km), Kolkata (1870 km)
Delhi -> Mumbai (1400 km), Jaipur (280 km), Lucknow (555 km), Hyderabad (1700 km)
Hyderabad -> Delhi (1700 km)
Jaipur -> Delhi (280 km), Ahmedabad (660 km)
Kolkata -> Varanasi (680 km), Bengaluru (1870 km)
Lucknow -> Delhi (555 km), Varanasi (320 km)
Mumbai -> Delhi (1400 km), Ahmedabad (530 km), Bengaluru (980 km)
Varanasi -> Lucknow (320 km), Kolkata (680 km)

LongestPathFinderApp

Longest Path Finder

DFS/backtracking longest simple path with Swing components

Route Search

Source City

Delhi

Destination City

Hyderabad

Find Longest Path

Longest Path: Delhi -> Hyderabad

Total Distance: 1700 km

Longest simple path found.

Add New City

City Name

Jaipur

Connect To

Hyderabad

Distance

2000

Add City

City Map

```

graph LR
    Delhi((Delhi)) --- 1700 km --- Hyderabad((Hyderabad))
    Delhi --- 1400 km --- Mumbai((Mumbai))
    Delhi --- 1870 km --- Kolkata((Kolkata))
    Delhi --- 280 km --- Jaipur((Jaipur))
    Delhi --- 555 km --- Lucknow((Lucknow))
    Hyderabad --- 1490 km --- Ahmedabad((Ahmedabad))
    Hyderabad --- 980 km --- Bengaluru((Bengaluru))
    Hyderabad --- 660 km --- Kolkata
    Mumbai --- 1400 km --- Delhi
    Mumbai --- 980 km --- Ahmedabad
    Mumbai --- 320 km --- Lucknow
    Ahmedabad --- 530 km --- Mumbai
    Ahmedabad --- 660 km --- Jaipur
    Bengaluru --- 1490 km --- Ahmedabad
    Bengaluru --- 980 km --- Mumbai
    Bengaluru --- 1870 km --- Kolkata
    Lucknow --- 555 km --- Delhi
    Lucknow --- 320 km --- Varanasi((Varanasi))
    Kolkata --- 680 km --- Varanasi
    Kolkata --- 1870 km --- Bengaluru
    Jaipur --- 280 km --- Delhi
    Jaipur --- 660 km --- Ahmedabad
  
```

Available Routes

Bengaluru -> Mumbai (980 km), Ahmedabad (1490 km), Kolkata (1870 km)

Delhi -> Mumbai (1400 km), Jaipur (280 km), Lucknow (555 km), Hyderabad (1700 km)

Hyderabad -> Delhi (1700 km)

Jaipur -> Delhi (280 km), Ahmedabad (660 km)

Kolkata -> Varanasi (680 km), Bengaluru (1870 km)

Lucknow -> Delhi (555 km), Varanasi (320 km)

Mumbai -> Delhi (1400 km), Ahmedabad (530 km), Bengaluru (980 km)

Varanasi -> Lucknow (320 km), Kolkata (680 km)

LongestPathFinderApp

Longest Path Finder

DFS/backtracking longest simple path with Swing components

Route Search

Source City

Ahmedabad

Destination City

Delhi

Find Longest Path

Longest Path: Ahmedabad -> Mumbai ...

Total Distance: 4935 km

Longest simple path found.

Add New City

City Name

Jaipur

Connect To

Hyderabad

Distance

2000

Add City

City Map

```

graph LR
    Ahmedabad((Ahmedabad)) --- 1490 km --- Bengaluru((Bengaluru))
    Ahmedabad --- 530 km --- Mumbai((Mumbai))
    Ahmedabad --- 660 km --- Jaipur((Jaipur))
    Bengaluru --- 1490 km --- Ahmedabad
    Bengaluru --- 980 km --- Mumbai
    Bengaluru --- 1870 km --- Kolkata((Kolkata))
    Mumbai --- 1400 km --- Delhi((Delhi))
    Mumbai --- 980 km --- Ahmedabad
    Mumbai --- 320 km --- Lucknow((Lucknow))
    Jaipur --- 280 km --- Delhi
    Jaipur --- 660 km --- Ahmedabad
    Kolkata --- 680 km --- Varanasi((Varanasi))
    Kolkata --- 1870 km --- Bengaluru
    Lucknow --- 555 km --- Delhi
    Lucknow --- 320 km --- Varanasi
    Varanasi --- 680 km --- Kolkata
  
```

Available Routes

Bengaluru -> Mumbai (980 km), Ahmedabad (1490 km), Kolkata (1870 km)

Delhi -> Mumbai (1400 km), Jaipur (280 km), Lucknow (555 km), Hyderabad (1700 km)

Hyderabad -> Delhi (1700 km)

Jaipur -> Delhi (280 km), Ahmedabad (660 km)

Kolkata -> Varanasi (680 km), Bengaluru (1870 km)

Lucknow -> Delhi (555 km), Varanasi (320 km)

Mumbai -> Delhi (1400 km), Ahmedabad (530 km), Bengaluru (980 km)

Varanasi -> Lucknow (320 km), Kolkata (680 km)

CONCLUSION

This project demonstrates the implementation of the Longest Path problem using efficient graph techniques. While shortest path algorithms like Dijkstra are widely applicable, longest path problems require careful constraints such as DAG structures to remain computationally feasible.

The visualization helps users understand how maximum-distance traversal differs fundamentally from shortest path logic. The project also emphasizes real-world applications such as project scheduling, critical path analysis, and network optimization.

REFERENCES

1. Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein.
MIT Press.

→ Covers graph algorithms, DAG, dynamic programming concepts.

2. Algorithm Design

Jon Kleinberg and Éva Tardos.
Pearson Education.

→ Includes longest path concepts and complexity discussions.

3. GeeksforGeeks

Articles on Longest Path in DAG and Graph Algorithms.

→ Practical implementations and examples.